

6. STACK ADTS

Stack concepts

Stack applications

Stack ADTs: requirements, contracts

Implementations of stacks: using arrays
and linked-lists

Stacks in the Java class library

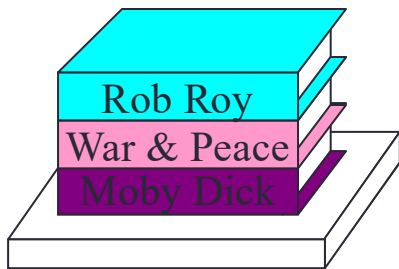
Stack concepts

- A **stack** is a last-in-first-out sequence of elements. Elements can added and removed only at one end (the **top** of the stack).
- We can **push** an element on to the stack, i.e., add it at the top of the stack.
- We can **pop** an element from the stack, i.e., remove it from the top of the stack.
- The **size** (or **depth**) of a stack is the number of elements it contains.

Example: stack of books

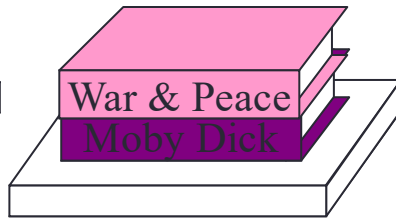
- Consider a stack of books on a table:

Initially:



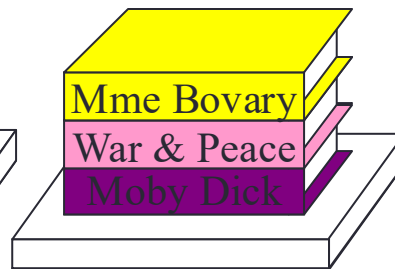
(size = 3)

After popping
a book:



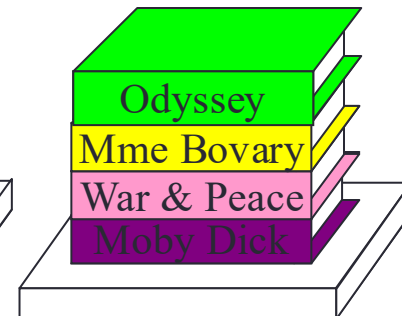
(size = 2)

After pushing
"Mme Bovary":



(size = 3)

After pushing
"Odyssey":



(size = 4)

- It is a stack because we can add (push) and remove (pop) books only at the top.

Stack applications

- Interpreter (e.g., Java Virtual Machine):
 - uses a stack to contain intermediate results during evaluation of complicated expressions
 - also uses the stack to contain arguments and return addresses for method calls and returns.
- Parser (e.g., XML parser, parser in Java compiler):
 - uses a stack to contain symbols encountered during parsing of the source code.

Example: text-file reversal

- A text file is a sequence of (zero or more) lines.
- To reverse the order of these lines, we must store them in a first-in-last-out sequence.
- Text-file reversal algorithm:

To make file *output* contain the lines of file *input* in reverse order:

1. Make *line-stack* empty.
2. For each *line* read from *input*, repeat:
 - 2.1. Push *line* on *line-stack*.
3. While *line-stack* is not empty, repeat:
 - 3.1. Pop a line from *line-stack* into *line*.
 - 3.2. Write *line* to *output*.
4. Terminate.

Example: bracketing (1)

- A phrase is **well-bracketed** if:
 - for every left bracket, there is a later matching right bracket
 - for every right bracket, there is an earlier matching left bracket
 - the sub-phrase between a pair of matching brackets is itself well-bracketed.
- Examples and counter-examples (math notation):
 - $s \times (s - a) \times (s - b) \times (s - c)$ ----- well-bracketed
 - $(-b + \sqrt{[b^2 - 4ac]}) / 2a$ ----- well-bracketed
 - $s \times (s - a) \times (s - b \times (s - c))$ ----- not well-bracketed
 - $(-b + \sqrt{[b^2 - 4ac]}) / 2a$ ----- not well-bracketed

Example: bracketing (2)

- Bracket matching algorithm:

To test whether *phrase* is well-bracketed:

1. Make *bracket-stack* empty.
2. For each symbol *sym* in *phrase* (scanning from left to right), repeat:
 - 2.1. If *sym* is a left bracket:
 - 2.1.1. Push *sym* on *bracket-stack*.
 - 2.2. Else, if *sym* is a right bracket:
 - 2.2.1. If *bracket-stack* is empty, terminate with false.
 - 2.2.2. Pop a bracket from *bracket-stack* into *left*.
 - 2.2.3. If *left* and *sym* are not matched brackets, terminate with false.
3. Terminate with true if *bracket-stack* is empty, or with false otherwise.

Stack ADT: requirements

- Requirements:
 - 1) It must be possible to make a stack empty.
 - 2) It must be possible to push an element on to a stack (i.e., add it at the top of the stack).
 - 3) It must be possible to pop the topmost element from a stack (i.e., remove it from the stack).
 - 4) It must be possible to test whether a stack is empty.
 - 5) It should be possible to access the topmost element in a stack without popping it.

Stack ADT: contract (1)

- Possible contract for *homogeneous* stacks (expressed as a Java generic interface):

```
public interface Stack<E> {  
    // Each Stack<E> object is a homogeneous stack  
    // whose elements are of type E.  
    ////////////////////////////////// Accessors //////////////////////////////////  
    public boolean isEmpty ();  
    // Return true if and only if this stack is empty.  
    public E peek ();  
    // Return the element at the top of this stack.
```

Stack ADT: contract (2)

- Possible contract (*continued*):

```
//////////////////// Transformers //////////////////////
```

```
public void clear ();
```

```
// Make this stack empty.
```

```
public void push (E it);
```

```
// Add it as the top element of this stack.
```

```
public E pop ();
```

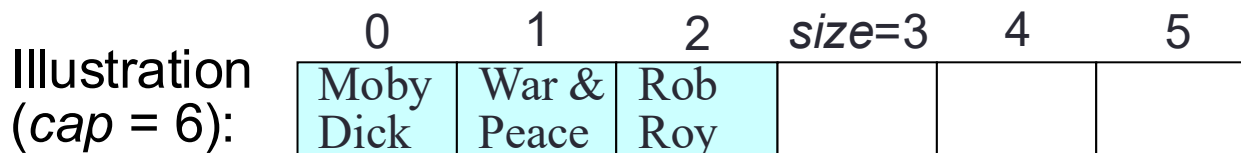
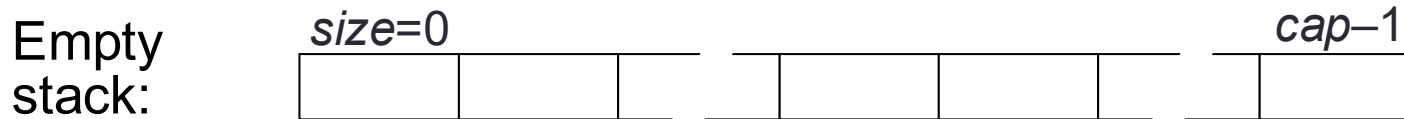
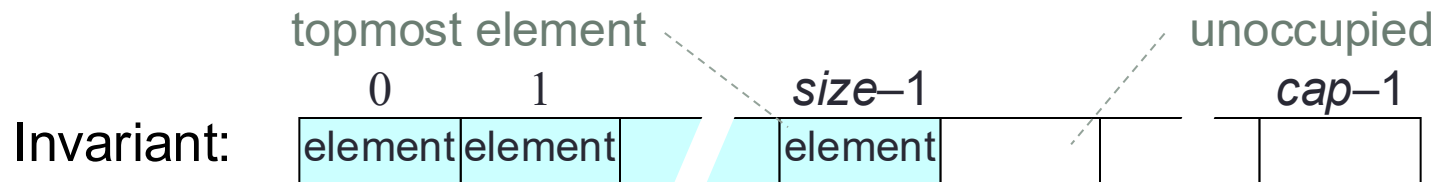
```
// Remove and return the element at the top of this
```

```
// stack.
```

```
}
```

Implementation of stacks using arrays (1)

- Represent a *bounded* stack ($\text{size} \leq \text{cap}$) by:
 - a variable *size*
 - an array *elems* of length *cap*, containing the elements in *elems*[0...*size*−1].



Implementation of stacks using arrays (2)

- Java implementation:

```
public class ArrayStack<E>
    implements Stack<E> {
    private E[] elems;
    private int size;
    ////////////////////////////////// Constructor //////////////////////////////////

    public ArrayStack (int cap) {
        elems = (E[]) new Object[cap];
        size = 0;
    }
```

Implementation of stacks using arrays (3)

- Java implementation (*continued*):

```
//////////////////// Accessors //////////////////////  
public boolean isEmpty () {  
    return (size == 0);  
}  
  
public E peek () {  
    if (size == 0)    throw ...;  
    return elems[size-1];  
}
```

Implementation of stacks using arrays (4)

- Java implementation (*continued*):

```
//////////////////// Transformers //////////////////////  
public void clear () {  
    size = 0;  
}  
  
public void push (E it) {  
    if (size == elems.length) ...  
    elems[size++] = it;  
}
```

--- The array is full.
Expand the array, or
throw an exception.

Implementation of stacks using arrays (5)

- Java implementation (*continued*):

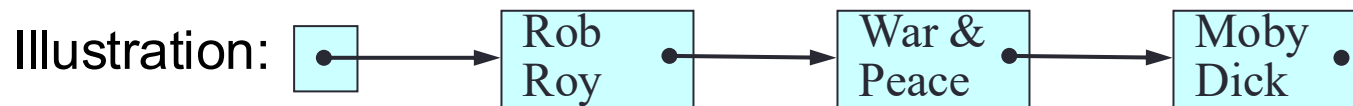
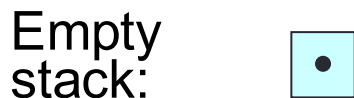
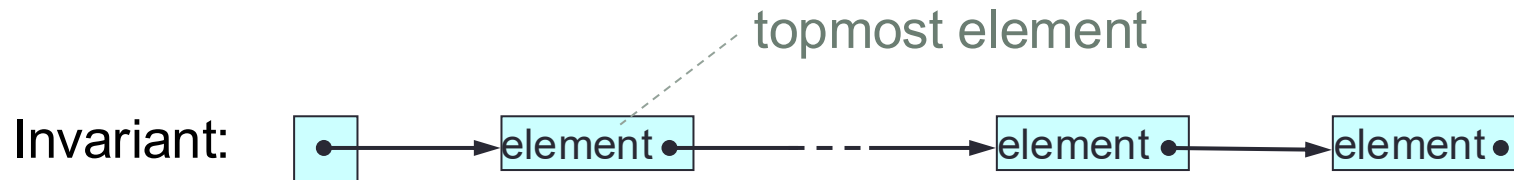
```
public E pop () {  
    if (size == 0)    throw ...;  
    E topElem = elems[--size];  
    elems[size] = null;  
    return topElem;  
}  
}
```

- Analysis:

- All operations have time complexity $O(1)$.

Implementation of stacks using SLLs (1)

- Represent an (unbounded) stack by an SLL, such that the first node contains the topmost element.



Implementation of stacks using SLLs (2)

- Java implementation (*continued*):

```

public class LinkedStack<E>
    implements Stack<E> {
    private Node<E> top;
    ////////////////////////////////// Inner class //////////////////////////////////
    private static class Node<E> {
        public E element;
        public Node<E> next;

        public Node (E x, Node<E> n) {
            element = x;  next = n;
        }
    }
}

```

Implementation of stacks using SLLs (3)

- Java implementation (*continued*):

```
//////////////////// Constructor //////////////////////
public LinkedStack () {
    top = null;
}

//////////////////// Accessors //////////////////////
public boolean isEmpty () {
    return (top == null);
}

public E peek () {
    if (top == null)    throw ...;
    return top.element;
}
```

Implementation of stacks using SLLs (4)

- Java implementation (*continued*):

```
//////////////////// Transformers //////////////////////  
public void clear () {  
    top = null;  
}  
  
public void push (E it) {  
    top = new Node<E>(it, top);  
}
```

Implementation of stacks using SLLs (5)

- Java implementation (*continued*):

```
public E pop () {  
    if (top == null)    throw ...;  
    E topElem = top.element;  
    top = top.next;  
    return topElem;  
}  
}
```

- Analysis:

- All operations have time complexity $O(1)$.

Stacks in the Java class library

- The library class `java.util.Stack<E>` is similar to the above class `ArrayStack<E>`.

- Illustration:

```
import java.util.*;  
Stack<Book> books = new Stack<Book>();  
books.push(moby_dick);  
books.push(war_and_peace);  
books.push(rob_roy);  
Book b = books.pop();
```

Example: text-file reversal again (1)

- Implementation of the text-file reversal algorithm:

```
public static void reverse (  
    BufferedReader input,  
    BufferedWriter output)  
    throws IOException {  
    // Make output contain the lines of input in reverse  
    // order.  
    Stack<String> lineStack =  
        new Stack<String>();  
    for (;;) {  
        String line = input.readLine();  
        if (line == null) break;    // end of input  
        lineStack.push(line);  
    }  
    input.close();  
}
```

Example: text-file reversal again (2)

- Implementation (*continued*):

```
while (! lineStack.empty()) {  
    String line = lineStack.pop();  
    output.write(line + "\n");  
}  
output.close();  
}
```

See code on Moodle. Shows how to do this using our own implementation of Stack, rather than Java's